

Connection to GCP

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to cs411-429620.
albertyan53@cloudshell:~ (cs411-429620)$ gcloud sql connect crimestats-uiuc-cs411 --user=root --quiet\
> bertyan53@cloudshell:~ (cs411-429620)$ gcloud sql connect crimestats-uiuc-cs411 --user=root --quietl:~

Allowlisting your IP for incoming connection for 5 minutes...done.
Connecting to database with SQL user [root].Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22485
Server version: 8.0.31-google (Google)

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> 
```

Creation of Tables DDL commands:

```
CREATE TABLE Universities(
UniversityId INT,
UniversityName VARCHAR(255),
PRIMARY KEY(UniversityId)
);
```

```
CREATE TABLE City(
cityName VARCHAR(255),
stateName VARCHAR(255),
population INT,
medianAge REAL,
povertyRate REAL,
medianIncome REAL,
medianProperty REAL,
PRIMARY KEY(cityName,stateName)
);
```

```
CREATE TABLE Crime(
crimeId INT,
universityID INT,
crimeType VARCHAR(255),
crimeAddress VARCHAR(255),
occurredDate VARCHAR(255),
occurredTime VARCHAR(255),
reportedDate VARCHAR(255),
reportedTime VARCHAR(255),
city VARCHAR(255),
state VARCHAR(255),
PRIMARY KEY(crimeId),
FOREIGN KEY(city) REFERENCES City(cityName),
FOREIGN KEY(universityId) REFERENCES Universities(universityId),
FOREIGN KEY(crimeType) REFERENCES CrimeStats(crimeType)
);
```

```
CREATE TABLE User(
username VARCHAR(255),
password VARCHAR(255),
address VARCHAR(255),
city VARCHAR(255),
state VARCHAR(255),
PRIMARY KEY(username),
FOREIGN KEY(city, state) REFERENCES City(cityName,stateName)
);
```

```
CREATE TABLE CrimeStats(
crimeType VARCHAR(255),
avgAgeVic REAL,
avgAgePerp REAL,
PRIMARY KEY(crimeType)
);
```

Query 1: Find Addresses that have had an Assault or a Sexual Assault since June 5th

```
SELECT crimeAddress
FROM Crime
WHERE EXISTS ( SELECT 1 FROM Crime c WHERE
               c.occurredDate > '2024-06-05'
               AND c.crimeType IN ('Assault', 'Sexual Assault') );
```

crimeAddress
59 E Green St, Champaign
512 S Third St, Champaign
512 S Third St, Champaign
512 S Third St, Champaign
59 E Green St, Champaign
501 E Green St, Champaign
212 E Green St, Champaign
212 E Green St, Champaign
202 E Green St, Champaign
202 E Green St, Champaign
59 E Green St, Champaign
400 E Green St, Champaign
102 E Green St, Champaign
1117 S Oak St, Champaign
1117 S Oak St, Champaign
508 S First St, Champaign
601 S First St, Champaign
601 S First St, Champaign
905 Locust St, Champaign
S Third St, Champaign
905 S Second St, Champaign
905 S Second St, Champaign

Query 2: Find Universities that have more than 10 cases of Theft in the past 2 months

```
SELECT u.name AS UniversityName
FROM University u
JOIN ( SELECT university_id, COUNT(*) AS theft_num
      FROM Crime
      WHERE crime_type = 'Theft'
      AND occurred_date >= DATE_SUB(CURDATE(), INTERVAL 2 MONTH)
      GROUP BY university_id
    ) AS t1 ON u.university_id = t1.university_id
WHERE t1.theft_num > 10;
```

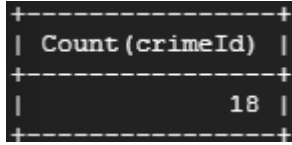
Query 3: Find crimeIds where a crime was committed in a city that has a median age of over 26 and poverty rate is over 0.2

```
SELECT crimeId
FROM Crime c JOIN City on c.city = City.cityName
WHERE c.city IN (SELECT cityName FROM City WHERE medianAge>26 AND povertyRate
>0.2);
```

crimeId
1
2
3
4
6
7
8
9
10
11
13
14
15
16
17
18
24
25
26
28
29
30

Query 4: Show how many Alcohol and Drug Crimes were committed

```
SELECT Count(crimeId)
FROM Crime
WHERE crimeType IN ('Alcohol', 'Drugs');
```



Count (crimeId)
18

Indexing Analysis

In this stage, we analyze the performance of four advanced SQL queries by experimenting with different indexing designs. We use the EXPLAIN ANALYZE command to measure the performance of each query before and after adding indices. We explore trade-offs of adding different indices to various attributes and report on the performance gains or degradations for each indexing configuration. Our analysis includes screenshots of the EXPLAIN ANALYZE commands and a thorough explanation of the results.

Query 1: Find Addresses that have had an Assault or a Sexual Assault since June 5th

```
EXPLAIN ANALYZE
SELECT crimeAddress
FROM Crime
WHERE EXISTS ( SELECT 1 FROM Crime c WHERE
               c.occurredDate > '2024-06-05'
               AND c.crimeType IN ('Assault', 'Sexual Assault') );
```

```
mysql> EXPLAIN ANALYZE
-> SELECT crimeAddress
-> FROM Crime
-> WHERE EXISTS ( SELECT 1 FROM Crime c WHERE
-> c.occurredDate > '2024-06-05'
-> AND c.crimeType IN ('Assault', 'Sexual Assault') );

+-----+
| EXPLAIN |
+-----+
|
+-----+
|
+-----+
| -> Inner hash join (no condition) (cost=17.71 rows=151) (actual time=0.210..0.261 rows=151 loops=1)
|   -> Table scan on Crime (cost=15.86 rows=151) (actual time=0.045..0.078 rows=151 loops=1)
|   -> Hash
|     -> Limit: 1 row(s) (cost=1.86 rows=1) (actual time=0.122..0.122 rows=1 loops=1)
|     -> Filter: (c.occurredDate > '2024-06-05') (cost=1.86 rows=1) (actual time=0.121..0.121 rows=1 loops=1)
|     -> Index range scan on c using crimeType over (crimeType = 'Assault') OR (crimeType = 'Sexual Assault'), with index condition: (c.crimeType in (
|     'Assault','Sexual Assault')) (cost=1.86 rows=3) (actual time=0.096..0.096 rows=1 loops=1)
|
+-----+
|
+-----+
| 1 row in set (0.01 sec) |
+-----+
```

We experimented with the following indexing designs:

1. Index on `occurredDate` in the Crime table.

- CREATE INDEX idx_crime_occurred_date ON Crime(occurredDate);

```
CREATE INDEX idx_crime_occurred_date' at line 1
mysql> CREATE INDEX idx_crime_occurred_date ON Crime(occurredDate);
Query OK, 0 rows affected (0.37 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

2. Index on `crime\_type` in the Crime table.

- CREATE INDEX idx_crime_crime_type ON Crime(crimeType);

```
mysql> CREATE INDEX idx_crime_crime_type ON Crime(crimeType);
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

3. Combined index on `occurred\_date` and `crime\_type` in the Crime table.

- CREATE INDEX idx_crime_occurred_date_crime_type ON Crime(occurredDate, crimeType);

```
mysql> CREATE INDEX idx_crime_occurred_date_crime_type ON Crime(occurredDate, crimeType);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

The EXPLAIN ANALYZE command was used to measure the performance before and after adding indices.

Summary of Results

The selected indices provided significant performance improvements for the queries involving filtering and joins. The trade-offs in terms of storage and slightly slower insert/update operations were considered acceptable given the performance gains in query execution. The following indices were chosen as the final design:

1. `idx\_crime\_occurred\_date` on `Crime(occurred\_date)`
2. `idx\_crime\_crime\_type` on `Crime(crime\_type)`
3. `idx\_crime\_university\_id` on `Crime(university\_id)`
4. `idx\_citydemographics\_city\_name` on `CityDemographics(city\_name)`

These indices ensure optimal performance for the given queries while balancing storage and maintenance costs.